Programmation Orientée Objet en Java

Projet Semestriel

« AGORA-ÉNERGIE »

Simulateur de Communauté Énergétique Locale et Marché Pair-à-Pair (P2P)

Raúl MAZO PEÑA

Année scolaire 2026-2027

Indications pour bien réussir votre projet

Recommandations pour la formation des groupes et l'organisation du travail

1. Formez des groupes de 4 étudiants, par affinité et par initiative propre.
2. Le mode Agile s'invite ici pour éviter l'effet tunnel et pour vous aider à livrer des versions exécutables dès le début du semestre.
3. Donnez-vous les moyens de travailler correctement : un bon IDE (ex. IntelliJ Ultimate), un outil de gestion de versions et de travail collaboratif (Git et GitHub), un outil de gestion de tâches (ex. Trello, GitHub Projects).
4. Les difficultés avec votre ordinateur, vos outils (performances, configuration, bugs, licences…) et votre équipe de travail (horaires, répartition des responsabilités, manque de confiance ou d'engagement, communication…) sont de la responsabilité de l'étudiant ou de l'équipe, et NON des enseignants.

Recommandations pour la réalisation du projet

5. Réfléchissez bien avant de coder : attention à la modélisation, au découpage en classes/paquets, à la duplication de code. Ceci est un cours de PROGRAMMATION ORIENTÉE OBJET et non de programmation impérative.
6. Testez, testez et… testez encore (mieux : écrivez des tests automatisés dès le début).
7. Rédigez une Javadoc complète pour toutes les classes, méthodes publiques et champs.
8. Relisez tout le code à tête reposée avant le rendu : cohérence entre classes, redondances, besoin de généricité, impact des évolutions futures.
9. Soumettez votre code à SonarQube et/ou Codacy pour une analyse statique de la qualité (code mort, bugs potentiels, non-respect des standards, commentaires manquants ou excessifs). Ces outils ne remplacent jamais les tests unitaires.

1. Contexte et objectifs du projet

La transition énergétique transforme la manière dont l'électricité est produite, stockée et consommée. Panneaux solaires résidentiels, bornes de recharge pour véhicules électriques, batteries domestiques et objets connectés font de chaque foyer un acteur potentiel du réseau — à la fois producteur et consommateur (« prosommateur »). Ce mouvement a fait émerger un nouveau type d'acteur économique : les plateformes numériques de communautés énergétiques locales (CEL), qui permettent à des voisins, des immeubles ou des quartiers d'échanger de l'énergie renouvelable produite localement, de manière transparente et automatisée, plutôt que de la revendre intégralement au réseau national.

Ce secteur (souvent appelé « energy sharing » ou « peer-to-peer energy trading ») est aujourd'hui un terrain fertile pour de nouvelles startups en Europe, à l'intersection de l'ingénierie logicielle, de l'économie de plateforme et de l'énergie. Votre projet consiste à concevoir et développer AGORA-ÉNERGIE, une application orientée objet en Java qui simule et gère une communauté énergétique locale : ses membres, leurs équipements, la production et la consommation dynamiques, et surtout un marché interne où l'énergie s'échange selon des règles métier réalistes.

L'application doit fonctionner comme un démonstrateur (Proof of Concept / MVP) crédible : quelque chose qu'une équipe pourrait présenter à des investisseurs ou à une collectivité comme base d'une future startup (motivation supplémentaire si vous avez l'esprit entrepreneurial). Comme pour les

projets des années précédentes, la validation des principes de la Programmation Orientée Objet (POO) doit se faire en deux temps : d'abord une interface en mode console (CLI) pour valider la logique métier, puis une interface graphique (GUI) en JavaFX pour la visualisation et l'interaction (avec la possibilité de l'utiliser dans au moins 2 langues européennes : français et une autre).

Propos de valeur du démonstrateur

- Permettre à des « prosommateurs » européens hétérogènes (résidentiel solaire, copropriété avec bornes de recharge, petite entreprise avec éolienne, etc.) d'échanger de l'énergie renouvelable entre eux plutôt que de dépendre uniquement du réseau public.
- Maximiser l'autoconsommation collective de la communauté et réduire la facture énergétique de ses membres.
- Offrir un marché interne transparent, avec des règles de tarification claires et auditables (historique des transactions).
- Respecter les contraintes physiques et réglementaires du réseau local (capacité, pertes, priorités de sécurité).

Utilisation responsable de l'IA générative

Contrairement à un exercice académique à l'ancienne, AGORA-ÉNERGIE combine plusieurs sous-domaines interdépendants : modélisation physique des équipements, règles contractuelles et tarifaires, contraintes de réseau, mécanique de marché (carnet d'ordres, appariement offre/demande) et simulation temporelle. Chaque décision de conception (comment modéliser un contrat ? comment gérer une transaction partiellement exécutée ? comment répartir l'énergie quand la demande dépasse l'offre ?...) exige une réflexion architecturale propre à votre équipe. Il n'existe pas de solution générique unique : c'est pour cela que je vous conseille de ne pas tomber dans la tentation de faire une génération directe et complète par un outil d'IA car le résultat n'aura pas le niveau requis. La manière dont ce projet a été conçu vous obligera donc à concevoir, débattre et justifier vos propres choix de conception en équipe.

2. Règles d'organisation et dates clés

Constitution des équipes

- Formez-vous en groupes de 4 étudiants, par initiative propre (aucune répartition ne sera faite par les enseignants, sauf cas exceptionnel de dernier recours).
- Chaque groupe nomme un(e) responsable / chef(fe) d'équipe, qui sera l'interlocuteur(trice) privilégié(e) des enseignants pour les questions logistiques.
- Le rôle de responsable ne dispense pas les autres membres de s'impliquer à parts égales dans la conception, le code, les tests et le rapport : la répartition du travail sera vérifiée en soutenance.

Déclaration d'équipe (fichier Excel)

Les/las responsables de chaque groupe doivent se mettre d'accord pour créer un fichier Excel partagé unique pour le cours avec l'information suivante :

- Le nom complet de chaque membre de l'équipe.
- L'adresse email institutionnelle de chaque membre.

- Le nom choisi pour l'équipe (facultatif, utile pour le canal Teams).
- Un choix de passage pour la démonstration/présentation/évaluation de chaque projet (30 min par groupe à placer entre le mercredi 6 janvier entre 8h et 12h30 et le jeudi 7 janvier entre 10h et 12h30).

Ensuite, il faudra partager le document complet avec l'enseignant (raul.mazo@ensta.fr) au plus tard deux semaines après le lancement du projet.

Objectif : ces informations serviront exclusivement à créer les canaux privés Microsoft Teams de chaque équipe, utilisés pour le suivi des avancées, les questions aux enseignants et le dépôt final du projet.

Canal Teams et suivi

- Chaque équipe disposera d'un canal privé Teams créé à partir du fichier Excel envoyé.
- Les avancées, questions et livrables intermédiaires et final doivent être partagés via ce canal.
- En cas de panne de Teams uniquement, contactez les enseignants par email avec un lien de téléchargement du projet et de la documentation (jamais en pièce jointe : les filtres SMTP bloquent le code source).

Jalons et dates clés

Jalon	Livrable attendu	Échéance indicative
J0 — Constitution des équipes	Fichier Excel de l'équipe (noms + emails institutionnels) envoyé par le·la responsable	Sous 2 semaines après le lancement du projet — date à confirmer en cours
J1 — Cadrage	Cahier des charges détaillé de l'équipe + premier diagramme de classes (avant de coder)	Fin du 1er mois du projet
J2 — Version CLI fonctionnelle	Noyau métier testé en mode Console, dépôt Git actif	Mi-parcours du semestre
J3 — Rendu final	Projet .zip + rapport complet + journal de prompts, déposé sur le Teams Channel de l'équipe	Dimanche 3 janvier 2027, minuit
J4 — Soutenances	Présentation (20 min) + questions (10 min) par équipe	6 et 7 janvier 2027

Les jalons J0 à J2 ne sont pas notés individuellement mais seront examinés lors des points d'avancement en cours ; ils conditionnent la qualité du travail final et sont fortement recommandés pour une approche Agile réussie.

3. Exigences techniques et architecture POO

3.1. Les membres de la communauté et leurs équipements

Classe abstraite *MembreCommunaute* : centralise les attributs communs à tout participant :

- Identifiant (ID), adresse / localisation dans le réseau, contrat associé, solde du compte, historique de consommation et de production.

- La gestion des données doit être rigoureuse (encapsulation), avec des méthodes de mise à jour de solde et d'historique qui valident les contraintes métier (ex. refus d'un solde négatif au-delà d'un seuil).

Héritage des profils : selon leur rôle dominant dans la communauté :

- ProsommateurResidentiel (hérite de MembreCommunaute) : foyer avec panneaux solaires et/ou batterie.
- ProsommateurProfessionnel (hérite de MembreCommunaute) : petite entreprise ou copropriété, volumes plus importants, contraintes contractuelles spécifiques.

Hiérarchie des équipements (au moins 10 classes)

Les étudiants doivent implémenter au moins 10 classes dans la hiérarchie des équipements pour démontrer la maîtrise de l'héritage (Equipement → SourceProduction → PanneauSolaire, etc.) :

- PanneauSolaire : production dépendante de l'ensoleillement et de la surface installée.
- Eolienne : production dépendante de la vitesse du vent (modèle simplifié).
- BatterieDomestique : capacité, état de charge, cycles de charge/décharge, dégradation.
- VehiculeElectriqueV2G : peut consommer (recharge) ou injecter de l'énergie au réseau local (Vehicle-to-Grid).
- AppliancePrographmable : appareil pouvant décaler sa consommation selon le prix (ex. lave-linge, chauffe-eau).

Contrats de comportement (interfaces)

Les comportements transversaux doivent être définis par des interfaces, dans l'esprit du Principe de Ségrégation des Interfaces (ISP) :

- Producteur : produireEnergie(instant), capaciteProduction().
- Consommateur : consommerEnergie(instant), besoinEnergie().
- Stockable : stocker(quantite), destocker(quantite), niveauStock().
- Negociable : proposerOffre(offre), accepterOffre(offre), retirerOffre(offre).
- Pilotable : activer(), desactiver(), reprogrammer(instant).

3.2. Modélisation du réseau et des contraintes

Classe *ZoneReseau* : représente le segment de réseau local partagé par la communauté, et doit gérer :

- Capacité maximale de transit (contrainte physique du transformateur local).
- Pertes en ligne, modélisées en fonction de la distance entre membres.
- Conditions environnementales dynamiques : ensoleillement, vitesse du vent, température, qui influencent la production des équipements à chaque pas de temps.
- Alertes de surcharge : toute transaction ou agrégation de flux dépassant la capacité doit être détectée et refusée ou renégociée.

La méthode Equipement.produire()/consommer() devra interroger la ZoneReseau pour obtenir les facteurs environnementaux actuels et ajuster la production, la consommation ou le déclenchement d'alertes.

3.3. Le marché de l'énergie (cœur du système)

Classe *MarcheEnergie* : gère un carnet d'ordres (offres de vente / demandes d'achat) et l'appariement (matching) des transactions.

- Types de tarification (Patron de conception Stratégie) : prix fixe communautaire, enchère au meilleur prix, tarification dynamique selon l'heure et l'équilibre offre/demande.
- Classe Transaction : horodatage, quantité, prix, membres impliqués, statut (PROPOSEE, ACCEPTEE, EXECUTEE, ANNULEE, REJETEE_RESEAU).
- Classe Contrat : types de contrats (ContratFixe, ContratVariable, ContratCommunautaire) avec règles de facturation propres, gérées par héritage ou composition selon le choix de conception de l'équipe.
- Gestion des exceptions métier : offre invalide, solde insuffisant, dépassement de capacité réseau, contrat expiré — chacune doit être une exception dédiée, gérée et documentée.

3.4. Gestion de la communauté et simulation

Classe *GestionnaireCommunaute* : orchestre la simulation dans le temps (pas de temps configurable : heure ou jour) et calcule des indicateurs de performance :

- Taux d'autoconsommation collective de la communauté.
- Économies réalisées par membre et par la communauté.
- Historique des transactions et des alertes réseau.
- Suggestion du membre le plus adapté pour absorber un excédent ou combler un déficit d'énergie (selon la disponibilité et la distance au réseau).

3.5. Interfaces utilisateur

10. Interface Console (CLI) : permet de créer des membres et équipements, lancer des pas de simulation, passer des offres sur le marché, consulter soldes et historiques — avant de passer à l'interface graphique.
11. Interface Graphique (JavaFX – GUI) : visualisation en temps réel de la ZoneReseau et des flux d'énergie, tableau de bord du marché (carnet d'ordres, transactions en cours), affichage des alertes (surcharge, panne d'équipement, contrat en défaut).

3.6. Extensions (points bonus)

- OptimiseurEchange : classe qui, selon le Patron Stratégie, propose l'appariement optimal entre offres et demandes (ex. minimiser les pertes réseau, maximiser l'équité entre membres).
- Prévision simple de production/consommation (heuristique ou modèle statistique basique) pour anticiper les besoins du lendemain.
- Sérialisation Java pour sauvegarder et recharger l'état complet de la simulation.
- Graphiques JavaFX pour visualiser l'évolution historique des prix, de l'autoconsommation ou de l'état des batteries.
- Simulation d'un incident (panne d'un équipement, cyberattaque sur le marché) et de la résilience de la communauté.

3.7. Liste de contrôle (checklist) pour l'implémentation

- Toutes les classes, méthodes publiques et champs sont documentés avec Javadoc ?

- Le projet a été construit de manière évolutive : Mode Console (CLI) d'abord, JavaFX (GUI) ensuite ?
- Le code respecte la règle DRY, avec une utilisation efficace de l'héritage pour mutualiser les fonctionnalités communes ?
- Le projet compte au moins 10 classes en relation d'héritage et 5 interfaces ?
- Le code respecte les principes SOLID, en particulier SRP et OCP ?
- Les tests unitaires couvrent au moins 90 % des fonctionnalités ?
- Le code respecte les conventions de nommage ?
- Les exceptions métier sont traitées de manière adéquate et spécifique ?
- Les données sont correctement encapsulées (modificateurs d'accès, getters/setters, immutabilité si pertinent) ?
- Le projet est portable via un gestionnaire de dépendances (Maven ou Gradle) ?

Pour vous former aux principes SOLID, DRY et autres (en Java), veuillez consulter le livre suivant disponible dans la médiathèque de l'école : Raúl Mazo & Gabriel Camargo. Modern Web Engineering with Spring Boot: How to Build Robust Java-based Web Applications Using Engineering Principles and Architectural Patterns. Amazon KDP, ISBN: 979-827-254-985-3, France, 05/11/2025. https://www.amazon.com/dp/B0FZSG596Y

4. Guide d'utilisation responsable de l'IA générative

L'IA générative (Claude, ChatGPT, Copilot, etc.) est autorisée et même encouragée dans ce projet — mais comme copilote de développement, jamais comme substitut à la conception et à la compréhension. L'objectif du cours reste de vous former à concevoir une architecture orientée objet solide ; un code que vous ne pouvez pas expliquer n'est pas un code que vous avez appris à écrire.

Usages encouragés

- Refactoriser un morceau de code déjà conçu par l'équipe pour améliorer sa lisibilité ou son respect de SOLID.
- Générer des squelettes de tests unitaires à partir d'une spécification que vous avez écrite.
- Rédiger ou améliorer la Javadoc à partir de votre code déjà fonctionnel.
- Explorer des alternatives architecturales (ex. « quels patrons de conception pourraient s'appliquer à ce problème de tarification ? ») pour nourrir une discussion d'équipe, pas pour copier la réponse telle quelle.
- Déboguer une erreur précise en fournissant le contexte du code et le message d'erreur.

Usages non autorisés

- Demander à une IA de générer l'architecture complète du projet (diagramme de classes, modèle de domaine) à partir de l'énoncé, sans passer par une réflexion et une décision d'équipe.
- Copier-coller des blocs de logique métier significatifs sans les comprendre ni pouvoir les justifier en soutenance.
- Soumettre un rapport ou une documentation rédigée par l'IA sans relecture critique et sans refléter le travail réellement effectué.

Journal des prompts (livrable obligatoire)

Chaque équipe tient un journal des prompts, sous forme de tableau, à inclure dans le rapport final, avec au minimum les colonnes suivantes :

- Date et membre de l'équipe.
- Outil d'IA utilisé.
- Prompt utilisé (résumé fidèle).
- Résumé de la réponse obtenue.
- Modifications apportées par l'équipe avant intégration, et justification.

Section de métacognition (livrable obligatoire)

Le rapport final doit inclure une courte section (1 à 2 pages) où l'équipe réfléchit collectivement à son usage de l'IA :

- Dans quels cas l'IA a-t-elle réellement fait gagner du temps ou amélioré la qualité ?
- Dans quels cas ses suggestions étaient-elles incorrectes, incomplètes ou mal adaptées à votre architecture, et comment l'avez-vous détecté ?
- Quelles décisions de conception avez-vous prises de manière autonome, sans assistance de l'IA, et pourquoi ?

Lors de la soutenance, les enseignants pourront interroger individuellement n'importe quel membre de l'équipe sur n'importe quelle partie du code, y compris les parties assistées par IA. Une incapacité à expliquer une partie du code, quelle que soit son origine, sera pénalisée.

5. Critères d'évaluation et livrables

Livrables à déposer sur le canal Teams de l'équipe

12. Projet Java complet, exporté en .zip, nommé avec les noms des étudiants de l'équipe.
13. Rapport écrit comprenant au minimum : page de garde avec l'équipe, cahier des charges, nouvelles fonctionnalités ajoutées au cahier de charges de base, architecture logicielle (diagramme de modules/composants et diagramme de classes), garanties de qualité (tests, couverture, éventuels algorithmes d'optimisation), méthode de travail (planning, répartition des responsabilités), journal des prompts et section de métacognition sur l'usage de l'IA.
14. Fichier PPTX pour la présentation orale et la démonstration : 20 minutes de présentation + 10 minutes de questions par équipe, pendant les deux dernières séances du cours.

Grille d'évaluation résumée

Critère	Pondération	Ce qui est évalué
Architecture POO (modélisation, héritage, interfaces, patrons de conception)	25 %	Diagramme de classes, cohérence du modèle, justification des choix
Fonctionnalités et règles métier implémentées	20 %	Conformité au cahier des charges, gestion des cas limites

Tests et qualité du code	15 %	Couverture ≥ 90 %, JUnit/SonarQube/Codacy, respect SOLID/DRY
Documentation (Javadoc, rapport)	10 %	Complétude et qualité de la Javadoc et du rapport écrit
Usage responsable de l'IA générative	10 %	Journal de prompts, section de métacognition, capacité à expliquer le code
Méthode de travail et organisation d'équipe	10 %	Agilité, répartition des rôles, usage de Git/GitHub
Présentation orale et démonstration	10 %	Clarté, respect du temps, réponses individuelles aux questions

Cette grille est indicative et pourra être affinée en séance ; elle sert de guide pour prioriser vos efforts tout au long du semestre.

Informations pratiques

Nombre d'étudiants par groupe : 4

Deadline du rendu : dimanche 3 janvier 2027 à minuit

Soutenances : 6 et 7 janvier 2027